

# Advanced Object Oriented Programming

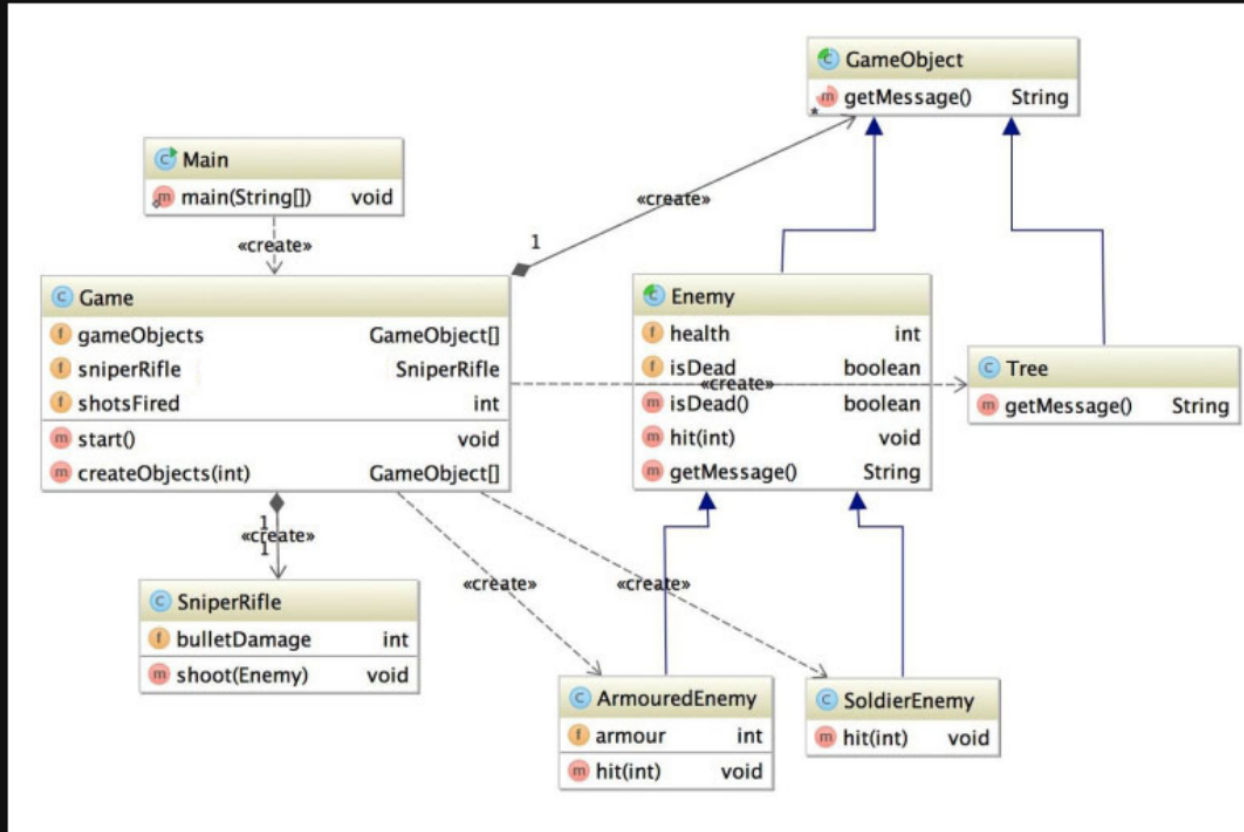
Created by <Academia de Código\_>

# Sniper Elite



The text-based version

# UML Class Diagram



# Game

- When creating the game specify the number of game objects;
- Create an array of game objects with enemies and trees;
- There is a higher probability of having enemies than trees;
- Iterate through the array and shoot only the enemies;
- Do NOT shoot the Trees!;
- Keep shooting the same enemy until it is dead;
- At the end show the total number of shots fired.

# Sniper Rifle

- When shooting, it has a probability of hitting the target with a certain damage amount.

# Enemies

- Soldier: it loses health on each hit;
- Armoured Soldier: if the armour has points it takes the damage, else the enemy loses health.

# { Exercise }

## Sniper Elite

Text based version

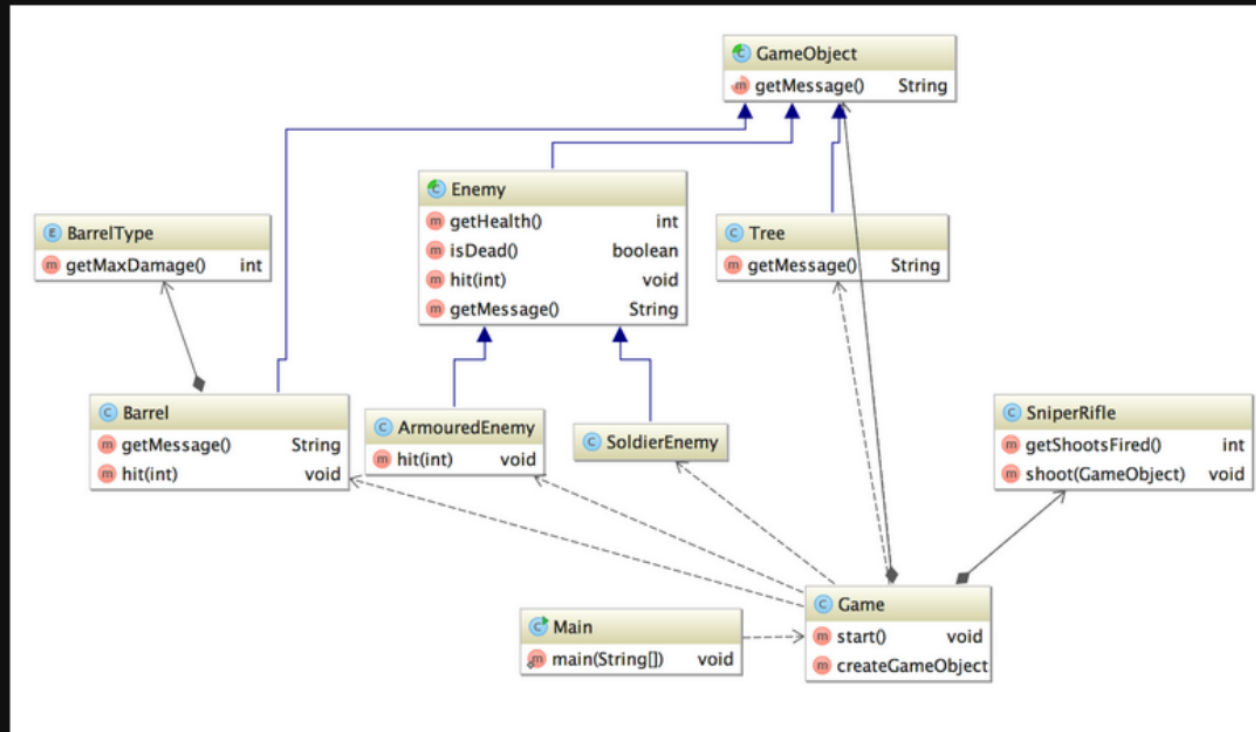


# What if...

- We want to shoot more than just enemies?
- Let's say we want to shoot some explosive barrels too;
- How should we organize our project classes?

# Explosive Barrels

Solution using Inheritance



# Solution using Inheritance

- Explosive barrels extend from GameObject;
- Change the shoot method parameter type to GameObject.

Our solution isn't ideal...

- Forces the implementation of the shoot method to check if the parameter is of type Barrel or Enemy;
- You have to come back and change it each time you add a hittable game object class.

# We want a way to...

- Use polymorphism to refer to classes that contain the hit method;
- Force those classes to correctly implement the hit method.

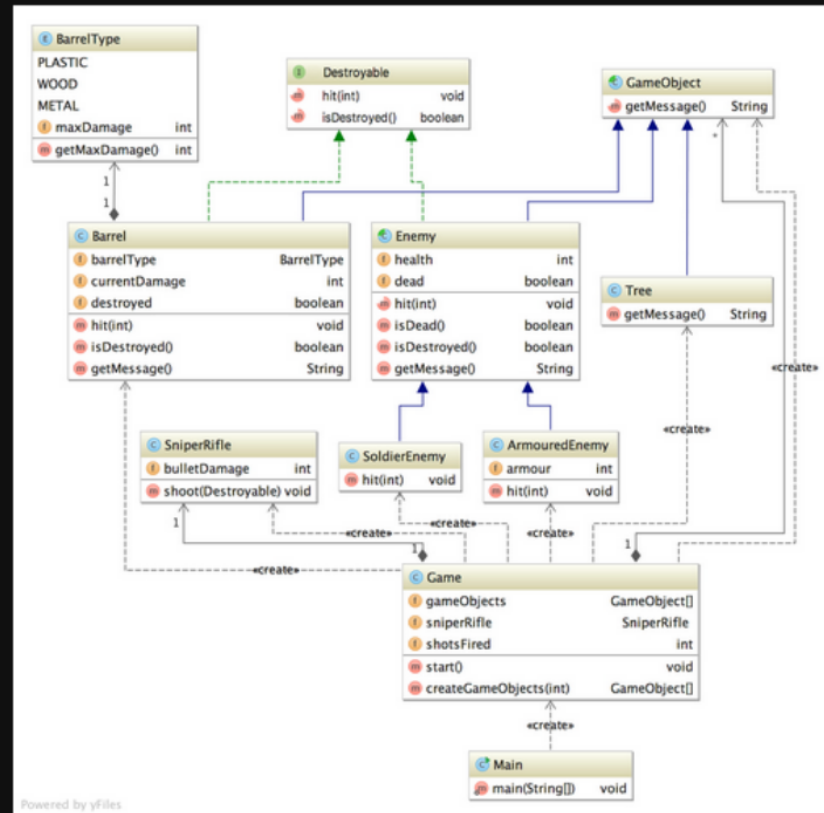
# Interfaces

An Interface is a completely abstract type that defines the behaviours a class must implement.

# Why Interfaces?

- Define a contract for the classes to follow;
- Take advantage of the polymorphism benefits;
- Not limited by the inheritance rigid structure.

# Interfaces to the rescue!



# Declare an Interface

```
/**
 * Destroyable.java
 */

// all methods are abstract
public interface Destroyable {

    public abstract void hit(int damage);

    public abstract boolean isDestroyed();

}
```

# Implement an Interface

```
/**
 * Barrel.java
 */
// when implementing an interface you need to override all methods
public class Barrel extends GameObject implements Destroyable {

    @Override
    public boolean isDestroyed() {
        // implement behaviour
    }

    @Override
    public void hit(int points) {
        // implement behaviour
    }
}
```

# Use the new Interface Type

```
public class SniperRifle {  
  
    private int bulletDamage = 1;  
    private final float HIT_PROB = 0.3f;  
  
    // declare a parameter of type Destroyable  
    public void shoot(Destroyable target) {  
  
        if(Math.random() < HIT_PROB) {  
  
            // call the method hit on the destroyable object  
            target.hit(bulletDamage);  
  
            System.out.println("It's a hit!");  
  
        } else {  
  
            System.out.println("It's a miss!");  
  
        }  
    }  
}
```

# { Exercise }

## Sniper Elite with Explosive Barrels

# Access Control

Feature that helps with encapsulation

# Access Level Modifiers

Determine classes, methods and fields access level.

| Modifier                      | Accessibility               |
|-------------------------------|-----------------------------|
| public                        | From everywhere             |
| protected                     | Same package and subclasses |
| default (aka package-private) | Same package                |
| private                       | Within the class            |

# Access Control Levels

| Level                       | Possible Modifiers                        |
|-----------------------------|-------------------------------------------|
| Top Level (aka class level) | public<br>default                         |
| Member Level                | public<br>protected<br>default<br>private |

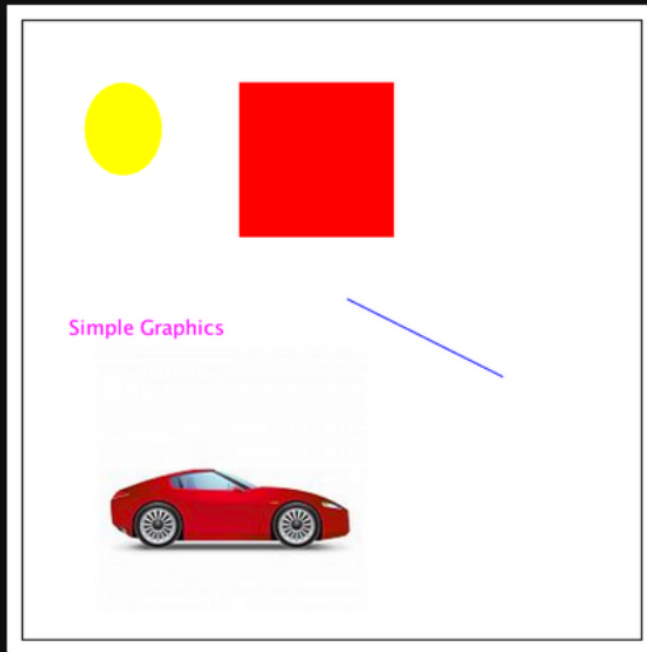
# How to decide the proper access control to use for members of your class?

Use the **most restrictive** access level possible.

**Avoid public** fields except for constants.

**Protected** and **default** might help with sub-classing.

# Simple Graphics Library



# Graphics

GFX classes representing drawable entities:

- Rectangles
- Ellipses
- Lines and Text
- Images



# Interfaces

Simple GFX classes implement one or more interfaces:

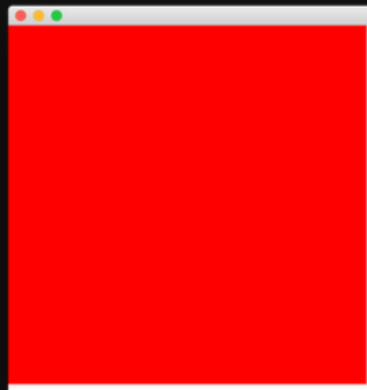
- Shape to draw and grow
- Movable to move shapes around
- Colorable to color shapes
- Fillable to paint shapes



# Canvas

Shapes are drawn to a canvas on a dedicated thread

```
Rectangle rect = new Rectangle(0, 0, 400, 400);  
rect.setColor(Color.RED);  
rect.fill();
```



The canvas will expand its size when required. The canvas always preserves a right and lower margin of 10px

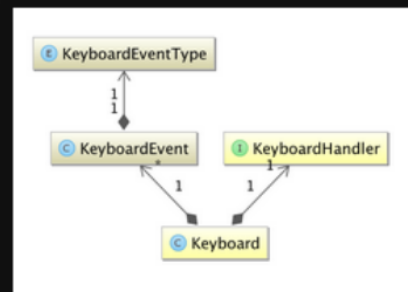
# Keyboard

The class that handles keyboard events should:

1. Implement the KeyboardHandler interface
2. Implement your logic inside KeyboardHandler methods

Your code should:

3. Instantiate a new Keyboard object with a KeyboardHandler reference
4. Instantiate as many KeyboardEvent objects as desired
5. Add KeyboardEvent objects to the Keyboard



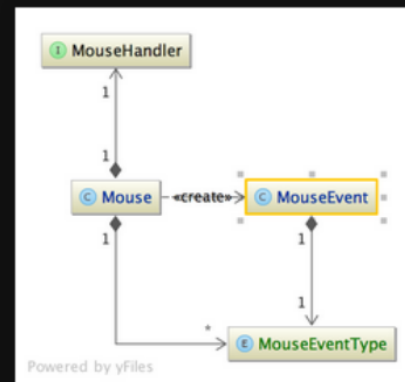
# Mouse

The class that handles mouse events should:

1. Implement the MouseHandler interface
2. Implement your logic inside MouseHandler methods

Your code should:

3. Instantiate a new Mouse object with a MouseHandler reference
4. Add MouseEventType objects to the Mouse



## { Exercise }

Test and Experiment with SimpleGFX library

# { Exercise }

## CarCrash Extended

Agnostic to Graphics Library



# { Hands On }

## Pong Skeleton

Agnostic to Graphics Library

# { Group Exercise }

## Roll your own game



# Exceptions

An exception is an abnormal event which occurs during the execution of a program that disrupts the normal flow of the program instructions

# Example

```
/**
 * Main.java
 */

public static void main(String[] args) {

    String[] names = new String[] { "Ze" };

    System.out.println("Hello " + names[1].toUpperCase());
}
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 1
at org.academiadecodigo.exceptions.Main.main(Main.java:14)
at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:39)
at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:25)
at java.lang.reflect.Method.invoke(Method.java:597)
at com.intellij.rt.execution.application.AppMain.main(AppMain.java:140)
```

Not catching an exception crashes the program.

# Catching an Exception

```
/**
 * Main.java
 */
public static void main(String[] args) {

    String[] names = new String[] { "Ze" };

    try { // try to run this

        System.out.println("Hello " + names[1].toUpperCase());

    } catch (Exception e) { // but if it fails run this code instead

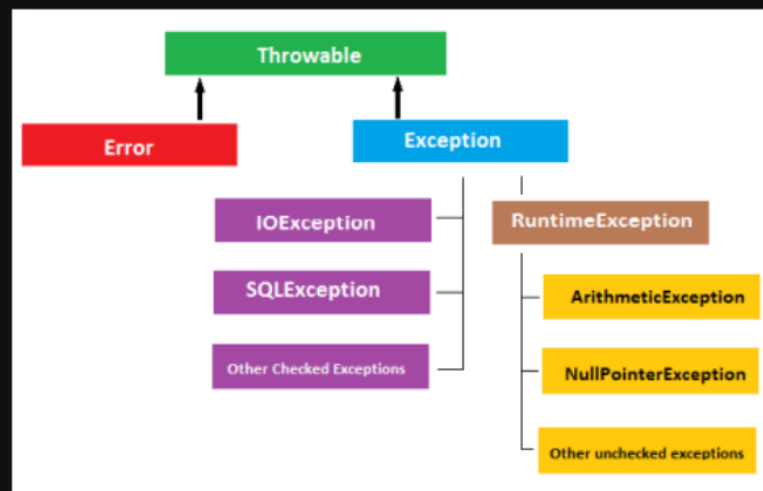
        System.out.println("Ze is not here");

    }

}
```

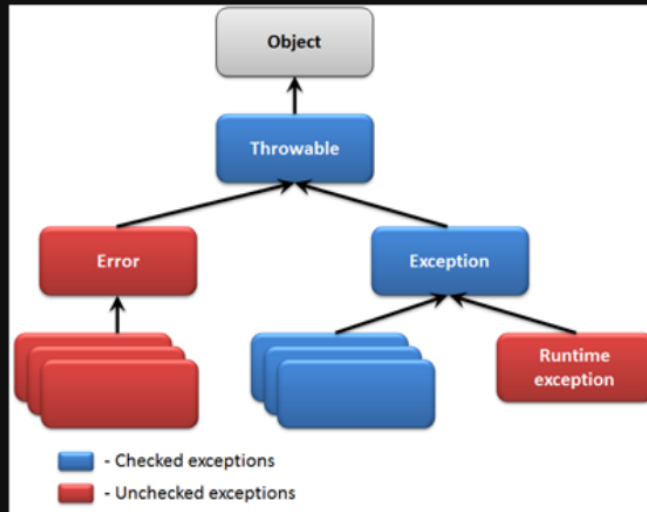
Please note that not all exceptions should be caught.

# Exceptions are objects



- Throwable: the super class of the error handling mechanism;
- Error: system errors that we don't need to catch;
- Exception: the ones we need to care about.

# Checked vs. Unchecked

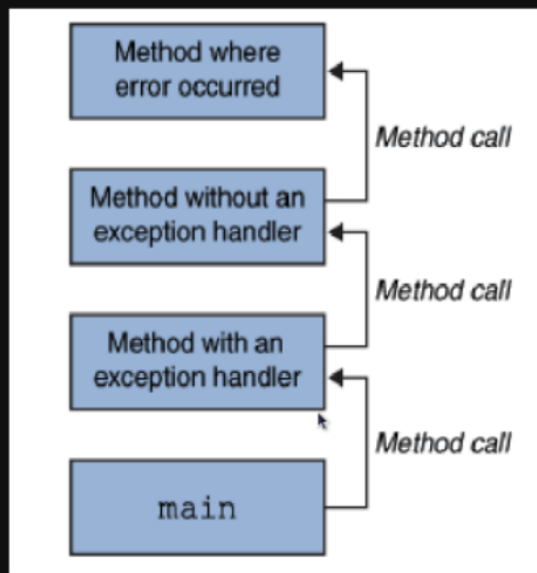


Only the checked exceptions should be caught.

# Chained Exceptions

If the caught exception cannot be handled correctly by the method, it can bubble up to another method

Some method up the call chain will catch the exception



# Exception Handling

```
public void method1() {  
    try {  
        method2();  
    } catch (Exception ex) {  
        doErrorProcessing(ex); // this method will handle the exception  
    }  
}  
  
public void method2() throws Exception{  
    method3();  
}  
  
public void method3() throws Exception {  
  
    // this method returns true if able to read the file  
    boolean readFile = readFile("VerySpecialFile");  
  
    if(!readFile) {  
        throw new Exception();  
    }  
}
```

When an exception is thrown we can choose to deal with it or pass it to the calling method

# The finally block

```
/**
 * Main.java
 */
public static void main(String[] args) {

    String[] names = new String[] { "Ze" };

    try { // try to run this

        System.out.println("Hello " + names[1].toUpperCase());

    } catch (Exception e) { // but if it fails run this code instead

        System.out.println("Ze is not here");

    } finally { // and this will always run

        System.out.println("Either way I will run whatever is in here...");

    }

}
```

The finally block always runs, even if the method is returning inside the try or catch.

# Catching multiple exceptions

```
// process of reading a file
try {
    open(file);
    determineSize(file);
    allocateMemory(memoryNeeded);
    read(file);
    close(file);
} catch (FileOpenException ex) {
    // something went wrong while opening the file
    handleOpenException();

} catch (SizeDeterminationException ex) {
    // something went wrong while determining the file size
    handleSizeDeterminationException();

} catch (MemoryAllocationException ex) {
    // something went wrong while trying to allocate memory
    handleMemoryAllocationException();

} catch (ReadException ex) {
    // something went wrong while trying to read the file
    handleReadException();

} catch (Exception ex) {
    // something went wrong with the file
    handleException(ex);
}
```

The less specific exceptions must be at the end.

# Creating your own exceptions

```
/**
 * A custom exception
 */
public class CustomException extends Exception {

    public CustomException() {
    }

    public CustomException(String s) {
        super(s);
    }
}
```

Usually the name of the exception should be self-explanatory, but a message can be added to provide additional information.

# Throwing your own exception

```
/**
 * Throwing a custom exception
 */
public class MyClass {

    public void someRiskyMethod() throws CustomException {
        if("something terrible happens") {
            throw new CustomException("Brace yourselves an exception is coming");
        }
    }
}
```

The method throws the exception when a specified condition occurs.

# Catching the exception

```
/**
 * Catching the custom exception
 */
public class Main {

    public static void main(String[] args) {

        MyClass myClass = new MyClass();

        try {
            myClass.someRiskyMethod();
        } catch (CustomException ex) {
            System.out.println(ex.getMessage()); // prints the exception message
        }

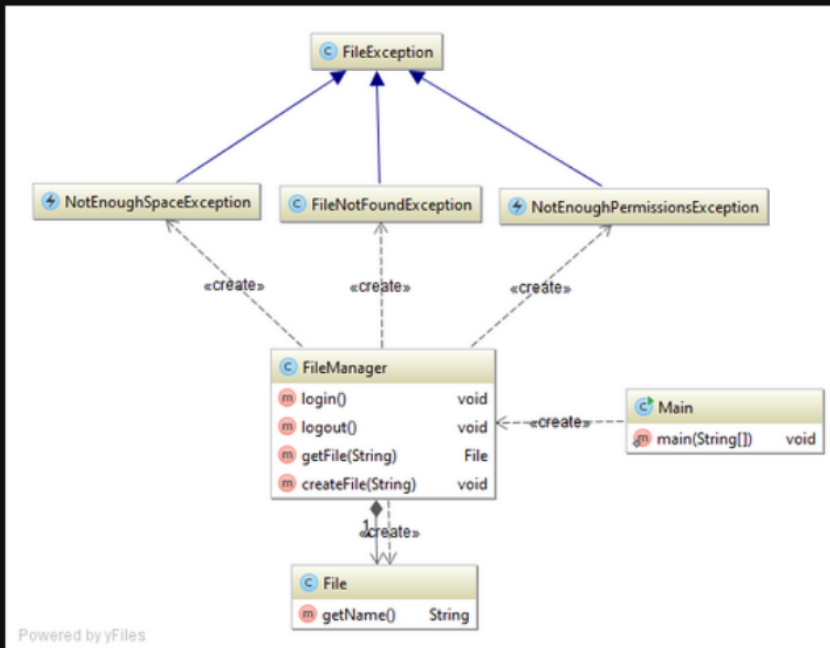
    }

}
```

The calling method handles the exception and deals with it.

# { Exercise }

## Exceptions



# Containers and Collections

# Containers

Used to hold any number of objects. Arrays are built in the Java programming language and have been our natural choice, but they have obvious limitations:



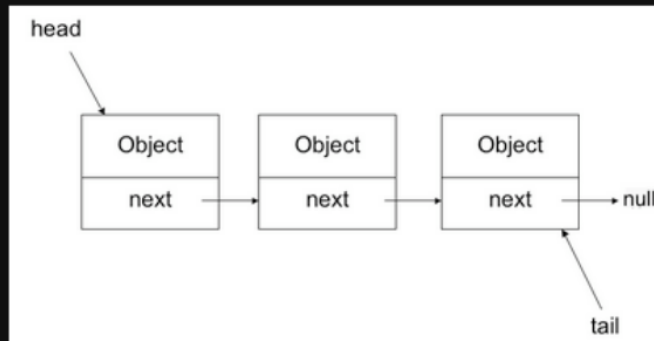
- Fixed size - the dimension of an array is determined the moment the array is created, and cannot be changed later on
- Memory usage - the array occupies an amount of memory that is proportional to its size, independently of the number of elements that are actually used
- Inefficient - sorting or changing the order of the elements requires moving elements around

What we need is a dynamic container!

# { Board Exercise }

## Dynamic Container

# Linked List



- data structure consisting of a group of nodes which together represent a sequence
- each node is composed of data and a reference to the next node in the sequence
- the number of nodes in a list is not fixed and can grow and shrink on demand

# { Board Exercise }

## Linked List

and the add method

# { Exercise }

## Linked List

Implement remaining methods until all tests pass

# Is our linked list Type Safe?

No. What does this mean?

- designed to hold `java.lang.Object`
- all classes can be upcasted to `Object` and stored in the container
- it is the programmer's responsibility to downcast the `Object` back to its original class
- the compiler is not able to check whether the downcasting is valid at compile-time
- but no problem, programmers are perfect in every way and never make mistakes



**{ Hands On }**

**Linked list type unsafety exercise**

# Generics

- pass type information to the compiler
- automatically insert the proper downcast operator
- ensures type safety at compile time



The type parameter section, delimited by angle brackets ( `<>` ) follows the class name and specifies the type variables

# Parameterized Types

The Box class wraps any type of object:

```
public class Box {  
    private Object object;  
    public void set(Object object) { this.object = object; }  
    public Object get() { return object; }  
}  
  
Box integerWrapper = new Box(new Integer(1)); // wrap an Integer  
String s = (String) integerWrapper.get(); // unsafe downcast produces run time error
```

We can make the Box class safer by using a type parameter:

```
public class Box<T> {  
    // T stands for "Type"  
    private T t;  
  
    public void set(T t) { this.t = t; }  
    public T get() { return t; }  
}  
  
Box<Integer> integerWrapper = new Box<>(new Integer(1)); // wrap an Integer  
String s = (String) integerWrapper.get(); // compile time error
```

# Type Parameters

A type parameter can be any non-primitive type:

- Class
- Interface
- Array
- Type Variable

By convention, type parameter names are single, uppercase letters.

The most commonly used type parameter names are:

- E - element
- K - key
- N - number
- T - type
- V - value
- S,U,V etc. - 2nd, 3rd, 4th types

**{ Hands On }**

**Linked list type safety exercise**

## { Exercise }

Modify Linked List to be type safe using Generics

# Generic Containers

Using generics, we can make our containers agnostic to the type of data they hold

```
LinkedList<Integer> integerList = new LinkedList<>();
LinkedList<String> stringList = new LinkedList<>();

integerList.add(new Integer(1));
integerList.add(new Integer(10));
integerList.add(new Integer(15));

stringList.add("a");
stringList.add("b");
stringList.add("c");

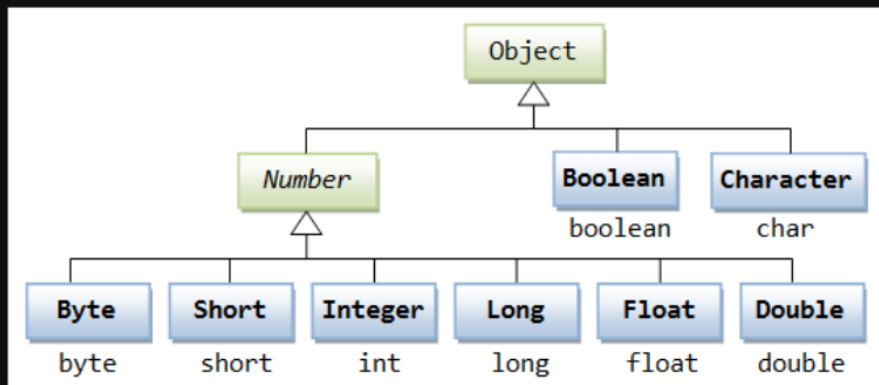
for (Integer i : integerList) {
    System.out.println(i);
}

for (String s : stringList) {
    System.out.println(s);
}
```

# Autoboxing & Unboxing

How can we store primitives in generic containers?

- a generic container contains only objects
- it cannot hold primitives such as int or double
- to put a primitive into a generic container a corresponding wrapper class has to be used



```
Integer intObj = 5566;    // autoboxing from int to Integer
int i = intObj;           // unboxing from Integer to int

Double doubleObj = 55.66; // autoboxing from double to Double
double d = doubleObj;     // unboxing from Double to double
```

# Iterating over Containers

The enhanced for syntax can be used to iterate over arrays

```
Integer[] integerArray = new Integer[3];
integerArray[0] = 1;
integerArray[1] = 10;
integerArray[2] = 15;

for (Integer i : integerArray) {
    System.out.println(i); // 1, 10, 15
}
```

You can use the same syntax to iterate over other container types, as long as they implement the Iterable interface

```
LinkedList<Integer> list = new LinkedList();

list.add(new Integer(1));
list.add(new Integer(10));
list.add(new Integer(15));

for (Integer i : list) {
    System.out.println(i); // 1, 10, 15
}
```

# Iterable Interface

Declares an abstract method called iterator() to retrieve an Iterator<E> object associated with the container

This Iterator object can be used to transverse through all the elements of its associated container

# Iterator Interface

Used to iterate over containers

Allows safe removal of elements during iteration

```
// Retrieve the Iterator associated with the container
Iterator it = list.iterator();

// Transverse this List using the Iterator
while (it.hasNext()) {

    // Retrieve and print each element
    System.out.println(it.next());

    if (someCondition()) {

        // Iterators allow safe removal while iterating
        it.remove();
    }
}
```

enhanced for loop can also be used to iterate through the container, but it lacks the remove capability

## { Exercise }

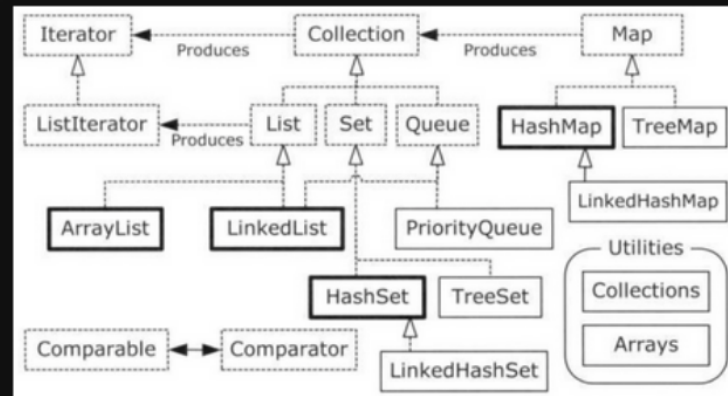
Implement an Iterable Integer Range class

# { Exercise }

## Iterable Linked List

Update your linked list to work with the enhanced for syntax

# Collections Framework - Java containers



- implemented in the package `java.util`
- four basic container interfaces

Map

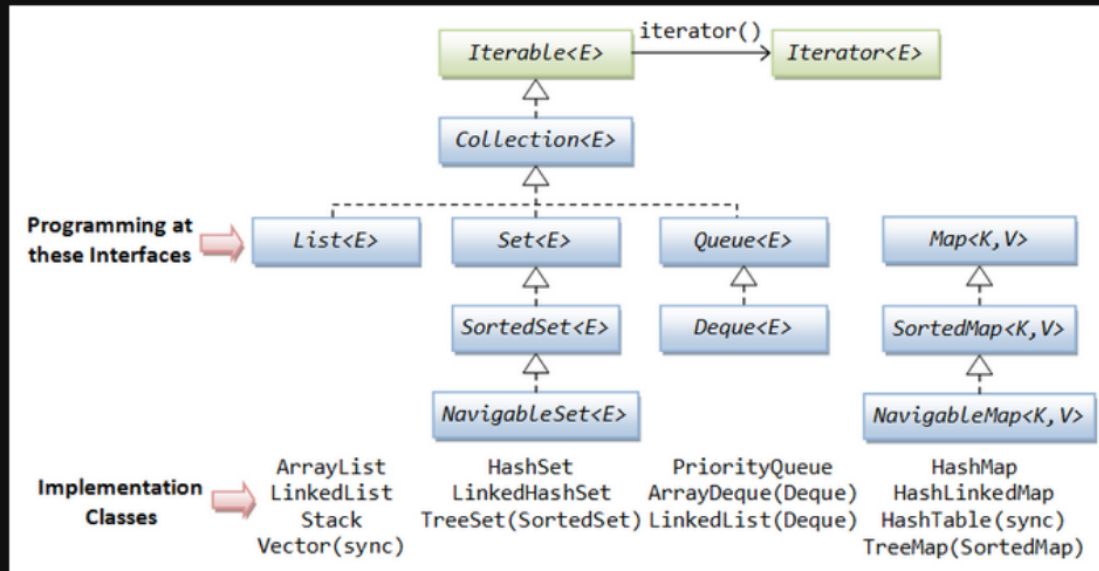
Set

List

Queue

# Support for Generics

All implementations of the collections framework use generic types



# Collection Interface

The root interface of the Collection Framework, abstracts the concept of a Group of Objects

Defines the common behaviours expected of all classes that belong to the Collections Framework

No assumptions are made about the order of the collection or whether it may contain duplicate elements

# Basic Collection Operations

```
// Returns the number of elements of the Collection
int size();

// Removes all the elements of the Collection
void clear();

// Returns true if there are no elements in the Collection
boolean isEmpty();

// Ensures that the Collection contains the given element
boolean add(E element);

// Removes the given element, if present
boolean remove(Object element);

// Returns true if the Collection contains the given element
boolean contains(Object element);
```

# Bulk Operations with other Collections

```
// Returns true if the Collection contains another Collection of any "unknown" type
boolean containsAll(Collection<?> c);

// Adds all elements of Collection of E or its subtypes to the Collection of E
boolean addAll(Collection<? extends E> c);

// Removes from the Collection all the elements contained in another Collection
boolean removeAll(Collection<?> c);

// Removes all elements from the Collection, retaining only the elements
// that are contained in another Collection
boolean retainAll(Collection<?> c);
```

? is a type wildcard

? extends E refers to the type of any class that extends or implements type E

# Iterating Collections

All Java collections are iterable

Iterators can be either fail-fast or fail-safe in their nature

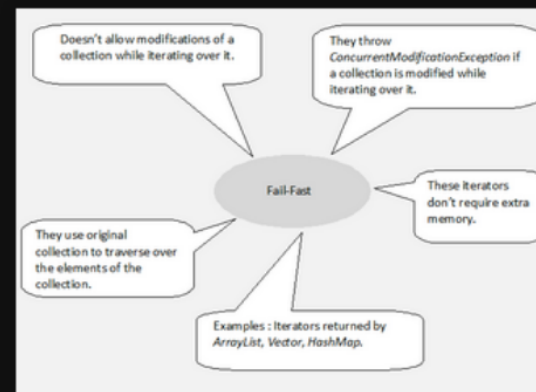
# Fail-Fast Iterators

The type of Iterator returned by most of the collections.

Will throw a `ConcurrentModificationException` if any structural modification happens to a collection while iterating over it.

Won't throw any exception if the collection is structurally modified by the iterator's own methods.

Better to fail than risking having the container in an unknown state.



# Fail-Safe Iterators

Won't throw any exception when a modification happens to a collection while iterating over it because they iterate over a clone of the collection.

It's not guaranteed that the data you're iterating on is up-to-date.

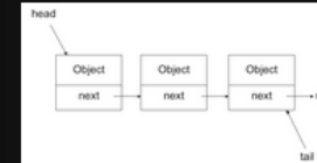
These iterators introduce additional overhead to our program in terms of time and memory (due to the creation of the collection copy).



# List interface

Holds a sequence of elements in the way they were inserted

- add() to insert objects
- get() to get them out one at a time
- iterator() to get an Iterator for the sequence.



## Implementations:

- ArrayList excels at randomly accessing elements, but is slower when inserting and removing elements in the middle of a List
- LinkedList provides optimal sequential access, with inexpensive insertions and deletions from the middle of the List. Relatively slow for random access, but it has a larger feature set than ArrayList, allowing it to be used as Queue or a Stack

# Collection Specific Iterators

```
List<String> lst = new ArrayList<>();  
lst.add("alpha");  
lst.add("beta");  
lst.add("charlie");  
  
ListIterator<String> iter = lst.listIterator();  
while (iter.hasNext()) {  
  
    // Retrieve each element  
    String str = iter.next();  
  
    // Duplicate each element  
    iter.add(str.substring(0));  
  
}
```

specific iterators like `ListIterator` may support additional functionality, like adding while iterating

# Collections Utilities

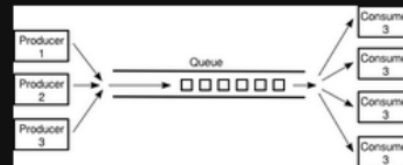
The `java.util.Collections` class provides useful functionalities for working with list collections, such as:

- copy all elements from a list into another list
- reverse the order of the elements in the list
- shuffle the list elements
- sort the list

# Queue interface

Produces elements in an order determined by a queuing discipline, usually FIFO. Especially important in concurrent programming, because they safely transfer objects from one thread to another.

- offer() to insert objects
- peek() to get the head of the queue without removing it
- poll() to retrieve and remove the head of the queue



## Implementations:

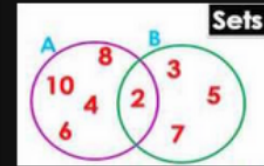
- LinkedList has methods to support queue behaviour and implements the Queue interface
- PriorityQueue objects are sorted into the queue, maintaining their natural order

## { Exercise }

TODO List using a Priority Queue

# Set interface

A Set can not have duplicate elements, it holds only one element of each value



- elements added to a Set must at least define equals() to establish object uniqueness.
- does not guarantee that it will maintain its elements in any particular order.

## Implementations:

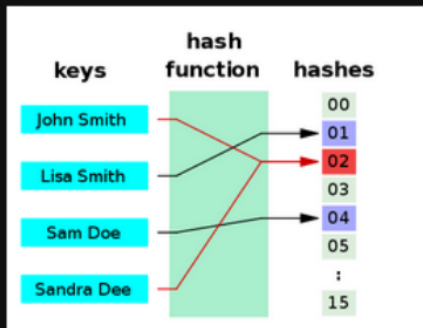
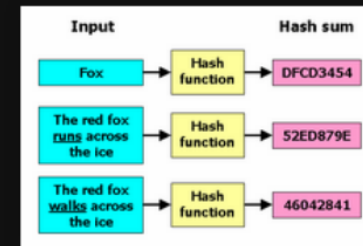
- HashSet uses hashing for speed and is optimized for rapid lookup. Elements must also define hashCode()
- TreeSet keeps elements in a sorted order. Elements must also implement the Comparable interface
- LinkedHashSet uses hashing for lookup speed, but maintain elements in insertion order using a linked list. Elements must also define hashCode()

## { Exercise }

UniqueWord class - Stores the unique words in a string,  
implements Iterable

# hash

A hash function is any function that can be used to map data of arbitrary size to data of fixed size.



A hash table is a data structure used in computer software for rapid data lookup. The hash table on the left utilizes a function that maps names to integers from 0 to 15.

There is a collision between "John Smith" and "Sandra Dee".

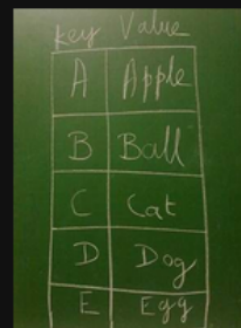
# Map interface

Look up an object using another object as key

- put() to insert key/object pairs
- get() to get the value for a specific key
- keySet() to get a Set view of all the keys in the map

Implementations:

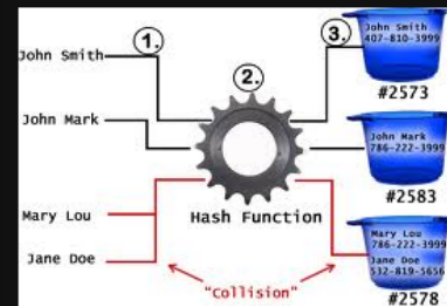
- HashMap provides near constant-time performance for inserting and locating pairs
- LinkedHashMap you get the pairs in insertion order when iterating
- TreeMap view the keys or the pairs in a sorted order determined by the Comparable interface



| key | Value |
|-----|-------|
| A   | Apple |
| B   | Bull  |
| C   | Cat   |
| D   | Dog   |
| E   | Egg   |

# Hash Map

- key/value pairs are stored in different buckets
- hashCode() of the key identifies the bucket
- equals() of the value identifies the element inside the bucket



hashCode – equals contract

- Objects that are equal according to the equals method must return the same hashCode value
- If two objects are not equal according to equals, they are not required to return different hashCode values

# hashCode() and equals()

The hash code can not depend on mutable data, as we risk having an object that produces different hash codes when it is put and get, making it impossible to retrieve from the map.

Default Object implementation uses the object address, which depending on the objects used as keys, might not work.

For a hash code to be effective, it must be fast and it must be generated based on the contents of the object.

Between hashCode() and equals(), the identity of the object must be completely resolved.

## { Exercise }

Class capable of producing a histogram of words in a string, implements Iterable

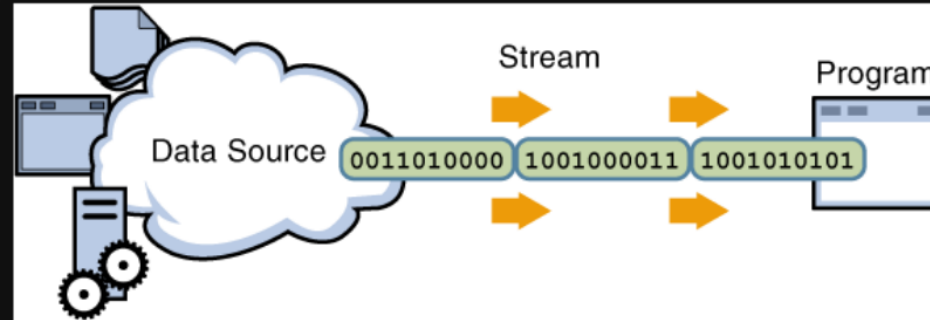
Perform 2 solutions, one using Inheritance, the other using Composition

[GFX NOT REQUIRED]

# Java I/O

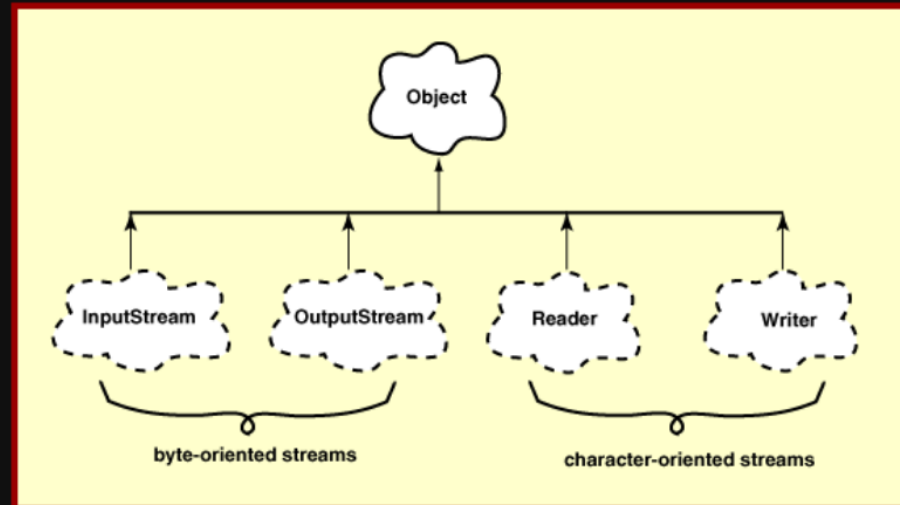
## Basic Input/Output Mechanisms

# I/O Streams



- A stream is a sequence of data
- Streams support many different kinds of data
- Some streams simply pass on data while others manipulate and transform it in useful ways
- A program uses an input stream to read data from a source, one item at a time
- A program uses an output stream to write data to a destination, one item at a time

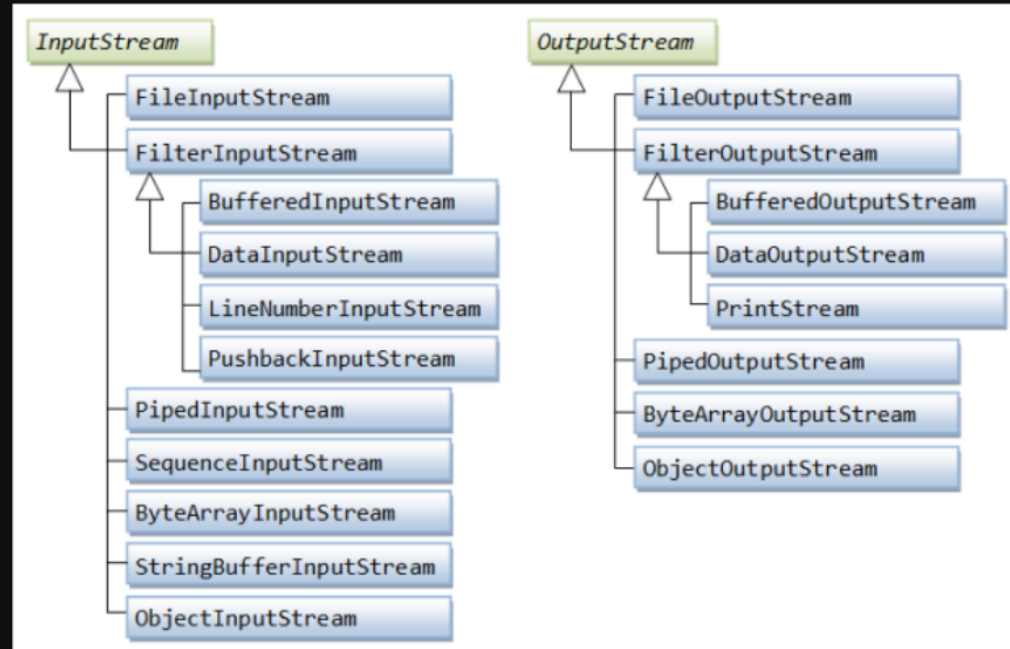
# Java I/O Streams



There are two types of streams in Java:

1. Byte streams used for handling byte input/output;
2. Character streams used for handling character input/output.

# Java Byte Streams



Byte streams perform input and output of 8-bit bytes.

# Reading bytes from a file

```
// open an input stream with a file path as the source
FileInputStream inputStream = new FileInputStream("file_path");

// read one byte from the file
int b = inputStream.read();

// test for end of file
if (b == -1) {
    // no more reading from this file...
}

// read multiple bytes from the file
byte[] buffer = new byte[1024];
int num = inputStream.read(buffer);

// print the number of bytes read
if (num != -1) {
    System.out.println("I have read this many bytes: " + num);
}

// don't forget to close the input stream
```

# Writing bytes to a File

```
// open an output stream with a file path as the destination
FileOutputStream outputStream = new FileOutputStream("file_path");

// write a single byte to file
byte b = 0x37;
outputStream.write(b);

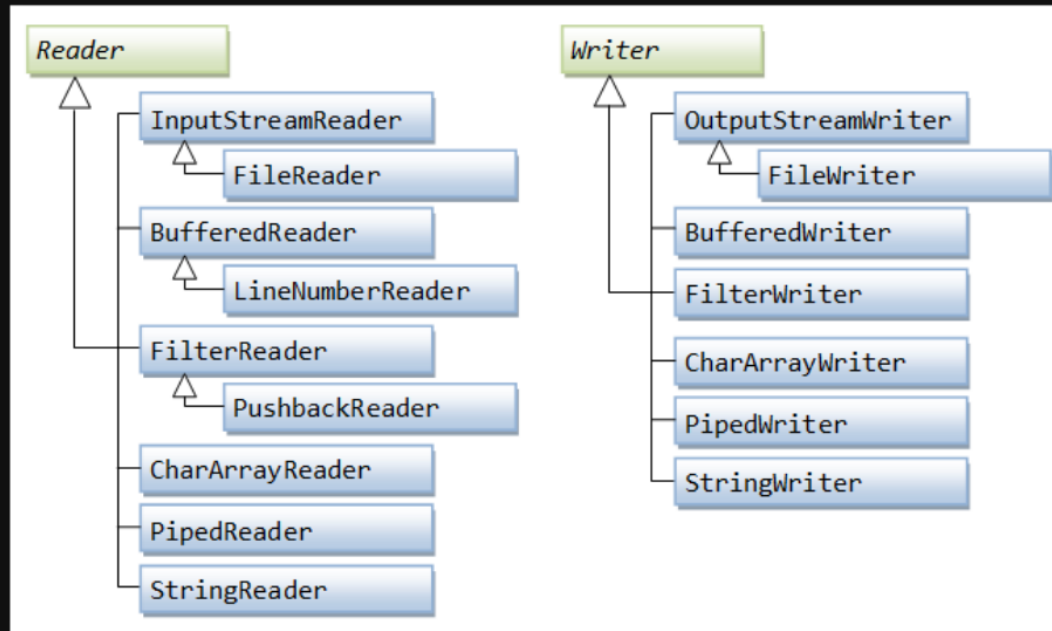
// write multiple bytes to file
byte[] buffer = "Some string".getBytes();
outputStream.write(buffer);

// don't forget to close the output stream
outputStream.close();
```

## { Exercise }

Create a class capable of copying the entire contents of a file

# Java Character Streams



Input and output performed with character stream classes are automatically translated to and from the specified character set.

# Character Sets

A defined list of characters recognized by computer hardware and software where each character is represented by a number

- US-ASCII Seven-bit ASCII, a.k.a. ISO646-US, a.k.a. the Basic Latin block of the Unicode character set
- ISO-8859-1 ISO Latin Alphabet No. 1, a.k.a. ISO-LATIN-1
- UTF-8 Eight-bit UCS Transformation Format
- UTF-16BE Sixteen-bit UCS Transformation Format, big-endian byte order
- UTF-16LE Sixteen-bit UCS Transformation Format, little-endian byte order
- UTF-16 Sixteen-bit UCS Transformation Format, byte order identified by an optional byte-order mark

# Character Streams

```
// FileReader is meant for reading streams of characters
FileReader fileReader = new FileReader(file);

// buffer to store decoded characters
char[] buffer = new char[1024];

// attempt to read the first 1024 characters from the file
// using the default character set to decode bytes into characters
int num = fileReader.read(buffer);

// print the number of characters read
if (num != -1) {
    System.out.println("I have read this many characters: " + num);
}

// construct a string from the buffer and print its content
String resultingString = new String(buffer);

// don't forget to close the input reader
fileReader.close();
```

# Buffered Reader

Reads text from a character-input stream in a buffered way, providing efficient reading of characters, arrays, and lines.

- Pre-fetches characters into a buffer for performance;
- Allows reading a file line by line instead of one character at a time.

# Buffered Reader

```
private String readFileByLine(String file) throws IOException {  
    // create a new file reader  
    FileReader reader = new FileReader(file);  
  
    // wrap the file reader using a buffered reader  
    BufferedReader bReader = new BufferedReader(reader);  
  
    String line = "";  
    String result = "";  
  
    // using the buffered reader we can read lines  
    while((line = bReader.readLine()) != null) {  
        result += line + "\n";  
    }  
  
    bReader.close();  
  
    return result;  
}
```

# Buffered Writer

Writes text from a character-input stream in a buffered way, providing efficient writing of characters, arrays, and lines.

- Characters are first written in a buffer;
- Buffer will then flush, writing a sequence instead of a character at a time;

# Buffered Writer

```
private String writeFileByLine(String file, String text) throws IOException {  
    // create a new file writer  
    FileWriter writer = new FileWriter(file);  
  
    // wrap the file writer using a buffered writer  
    BufferedWriter bWriter = new BufferedWriter(writer);  
  
    //add text to buffer  
    bWriter.write(text);  
  
    bWriter.flush(); // if the buffer is not full, flush will force disk write  
    bWriter.close(); // auto-flush is done on close  
}
```

# Additional Classes

- Print Writer - converts data into formatted output
- Data Streams - binary I/O of primitive data types
- Object Streams - binary I/O of objects (serialization)

# { Exercise }

## Read words from a text file

Create a WordReader class that implements iterable